# PodiumHQ — Hackathon Scoring Platform

DevSecOps Capstone Project | Final Year Technical Presentation

## What is PodiumHQ?

- A full-stack web platform for organizing and judging hackathons.
- Allows judges to evaluate teams across multiple rounds and weighted criteria.
- Computes team rankings in real time and displays an automated live leaderboard.

## Why I Built This (Problem & Goal)

- Solves the mess of manual spreadsheets and delayed scores during hackathon deadlines.
- Original application domain—built from scratch with zero TaskBoard code reuse.
- Goal: Take a real web app through the complete DevSecOps pipeline from commit to cloud.

## Student & Project Info

- Student Name: D.N.Nanda kishore
- Roll Number: 10224 | BATCH: A
- Stack: React 18, FastAPI, PostgreSQL, Docker, GitHub Actions, Terraform, Kubernetes

# Application Stack & Why I Chose These Tools

Simple, clean 3-tier architecture built with modern open-source tools

## Frontend: React 18 + Vite

- What it does: Single-page application with Tailwind CSS in an Obsidian & Solar Gold theme.
- Why I chose it: Needed a fast, reactive UI where judges can submit scores without page refreshes.
- Runtime: Served through unprivileged Nginx on port 8080 (running as UID 101).

## Backend: FastAPI (Python 3.12)

- What it does: Handles 15+ REST endpoints for teams, criteria, judges, and score calculations.
- Why I chose it: Automatic Swagger docs at /docs, async performance, and built-in health probes.
- Runtime: Hardened Python container running strictly as non-root appuser (UID 10001).

## Database: PostgreSQL 16

- What it does: Stores events, rounds, criteria, teams, judges, and evaluations across 6 tables.
- Why I chose it: Strong relational integrity, foreign keys, and safe schema migrations with Alembic.
- Connection: Managed with SQLAlchemy 2.0 ORM and Psycopg 3 connection pooling.

# System Architecture: From Code to Deployment

How code moves from local development to automated deployment and monitoring

| 1. Developer | 2. GitHub Actions | 3. Registry | 4. GitOps | 5. AWS EKS | 6. Monitoring |
|---|---|---|---|---|---|
| Git Push / Main | Lint -> Tests -> Scan | GHCR (Git SHA Tag) | ArgoCD Auto-Sync | VPC + Pods + HPA | Prometheus & Grafana |

## Delivery Flow (CI/CD to Registry)

- Developer pushes code to GitHub repository.
- GitHub Actions automatically runs linters, 31 pytest tests, and builds container images.
- Trivy scans images for security vulnerabilities before anything is published.
- Images are tagged with the Git commit SHA (e.g. :f990516) and pushed to GHCR.

## Runtime Flow (Cluster & Monitoring)

- ArgoCD detects new image tags in Git and synchronizes manifests with the EKS cluster.
- Ingress routes incoming traffic: / to frontend pods and /api to backend pods.
- Horizontal Pod Autoscaler (HPA) scales backend pods between 2 and 6 replicas based on CPU.
- Prometheus continuously scrapes /metrics to track response times and error rates.

# CI/CD Pipeline with GitHub Actions

Automating code checks, test coverage, and security vulnerability scanning

## Step 1: Code Quality & Linting

- What it does: Runs Ruff for Python code formatting and Mypy for static type validation.
- Why it matters: Catches syntax errors, bad imports, and type mismatches before tests run.
- Frontend: Runs npm run lint to catch unused variables and component bugs.

## Step 2: Automated Tests & Coverage Gate

- What it does: Executes 31 unit and integration tests using Pytest.
- Coverage Gate: Pytest-cov enforces a strict 80% minimum coverage threshold.
- Current Result: 90.99% overall code coverage. If coverage falls below 80%, the build fails.

## Step 3: Build & Security Scanning (Trivy)

- What it does: Builds multi-stage Docker images and scans them with Trivy.
- Security Gate: Fails the pipeline if any HIGH or CRITICAL CVEs are detected.
- Publishing: Successful builds are pushed to GHCR with immutable Git SHA tags, never :latest.

# Infrastructure as Code: Terraform on AWS

Writing code to provision cloud networks and Kubernetes instead of manual setup

## Network: AWS VPC (vpc.tf)

- Spans 3 Availability Zones in AWS ap-south-1 (Mumbai) for reliability.
- Creates 3 public subnets (for internet ingress) and 3 private subnets (for worker nodes).
- Includes a NAT Gateway so private worker nodes can safely pull container images.
- Tagged with kubernetes.io/role/elb so AWS can automatically manage load balancers.

## Compute: EKS Cluster (eks.tf & addons.tf)

- Deploys a managed Kubernetes v1.30 control plane with managed node groups.
- Worker nodes: 2 to 4 t3.medium EC2 instances distributed across private subnets.
- Add-on 1: AWS EBS CSI driver—required so PostgreSQL can attach persistent disk volumes.
- Add-on 2: Metrics Server—required so the Horizontal Pod Autoscaler can read CPU usage.

## Honest Cloud Status & Teardown Runbook

- Status: Terraform files are written, formatted, and validated with terraform validate.
- Cost Note: EKS and NAT Gateways cost ~$0.15/hour, so this was not left permanently running on AWS.
- Safety: Provided a complete terraform destroy runbook in terraform/README.md for clean teardown.

# Kubernetes & Helm: Packaging Workloads

Managing application deployments, services, scaling, and database migrations

## Workloads & High Availability

- Packaged as a Helm chart in helm/podiumhq with values-dev and values-prod.
- Backend & Frontend Deployments: 2 replicas each with RollingUpdate strategy.
- PodDisruptionBudget (PDB): Guarantees at least 1 backend pod stays online during node updates.
- Horizontal Pod Autoscaler (HPA): Automatically scales backend from 2 to 6 pods when CPU > 60%.

## Container Security in Kubernetes

- Enforces securityContext on all pods: runAsNonRoot: true.
- Backend runs as UID 10001; Frontend runs as Nginx UID 101.
- Drops all Linux capabilities (capabilities: drop: [ALL]) to limit attack surface.
- Ingress routes / to frontend, and /api, /health, /ready, /metrics to backend.

## Database Migration Job Hook

- The Problem: When 2+ backend pods boot at the same time, they race to run database migrations.
- The Solution: Created a Helm pre-upgrade/pre-install Job hook (migration-job.yaml).
- Alembic migrations run to completion first in an isolated job; application pods start only after it succeeds.

# GitOps with ArgoCD: Declarative Delivery

Using Git as the single source of truth for cluster state and self-healing

## What is GitOps & Why ArgoCD?

- Concept: Instead of engineers running manual kubectl apply, Git holds the desired state.
- ArgoCD controller runs inside the cluster and constantly compares live state against Git.
- Configured in argocd/application.yaml watching the helm/podiumhq chart in this repo.

## Automated Sync & Pruning

- Automated Sync: When CI updates the image tag in Git, ArgoCD automatically applies the change.
- Automated Pruning (prune: true): Removes old deleted resources from the cluster automatically.
- Audit Trail: Every cluster change corresponds to a verifiable Git commit hash.

## Self-Healing Drift Demonstration

- What is drift?: When someone manually alters resources in the cluster using kubectl.
- Self-Healing (selfHeal: true): ArgoCD detects the discrepancy and immediately reverts it.
- Demo check: If I run 'kubectl scale deployment backend --replicas=5', ArgoCD forces it back to 2.

# Observability: Prometheus & Grafana

Monitoring system health, request traffic, and custom judging metrics

## System Metrics on /metrics

- What it does: Backend exposes a Prometheus-format endpoint at /metrics.
- HTTP Telemetry: Tracks total requests, response status codes (2xx, 4xx, 5xx), and p95 latency.
- Runtime Telemetry: Tracks Python garbage collection, memory usage, CPU time, and open file descriptors.

## Custom Domain Business Metrics

- podiumhq_scores_submitted_total: Counter tracking how many judge scores have been submitted.
- podiumhq_teams_by_status: Gauge showing active vs evaluated teams in the hackathon.
- podiumhq_leaderboard_requests_total: Counter tracking how often the leaderboard is viewed.

## Grafana Dashboard & Alert Rules

- Grafana JSON (monitoring/grafana-dashboard.json): 5 panels showing RPS, p95 latency, 5xx errors, and scores.
- PrometheusRule (monitoring/alerts.yaml): Triggers an alert if 5xx errors exceed 1% for 2 minutes.
- Target Down Alert: Alerts immediately if the backend service stops responding.

# Testing, Security & Real Debugging Challenges

Verified test coverage, non-root checks, and authentic bugs I ran into and fixed

## Verified Test & Security Results

- Pytest Results: 31 tests passed in 1.32 seconds with 90.99% statement coverage.
- Test Isolation: Runs against an in-memory SQLite database so tests never pollute real data.
- Verified Non-Root: Ran 'docker compose exec backend id' -> uid=10001(appuser) and frontend -> uid=101(nginx).

## Real Bug 1: In-Container Pytest Permission

- The Problem: Running pytest inside the backend container crashed with sqlite3.OperationalError.
- Root Cause: Working directory /app was created by Docker as root; appuser couldn't write the .coverage file.
- The Fix: Added explicit 'RUN chown -R appuser:appgroup /app' in the Dockerfile.

## Real Bug 2: Nginx IPv6 Healthcheck Failure

- The Problem: Docker Compose reported the frontend container as 'unhealthy' even though web pages loaded.
- Root Cause: BusyBox wget resolved localhost to IPv6 ::1, but Nginx was only listening on IPv4 8080.
- The Fix: Added 'listen [::]:8080 default_server;' to the Nginx template and updated probe to 127.0.0.1.

# Summary, Limitations & Key Learnings

Honest self-assessment of achievements, cloud limitations, and future improvements

## What Works & Was Verified

- Fully functional hackathon scoring platform running locally on Docker Compose.
- 31 automated tests passing with 90.99% coverage and isolated test fixtures.
- Non-root container execution verified live on both backend and frontend.
- Complete, validated manifest suite: Terraform, Helm chart, ArgoCD, and Prometheus.

## Honest Limitations (Cloud Scope)

- AWS EKS & ArgoCD were not deployed to live cloud infrastructure to avoid ongoing student cloud costs.
- All cloud manifests are written, syntactically validated, and ready for deployment.
- Everything runnable locally was verified 100% green with saved evidence files.

## Top Engineering Lessons

- DevSecOps is in the details: file permissions, non-root users, and dual-stack networking.
- Helm pre-upgrade hooks are crucial to prevent database migration locks across replicas.
- Immutable image tags (Git commit SHAs) prevent deployment caching confusion.